PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

TALLER INVESTIGATIVO
ARQUITECTURAS DE SOFTWARE

Presentado a: **Instructora Alexandra Guevara**

Por Aprendiz: **Cristian David Yalanda**

Ficha: 3312932

Grupo: **ADSO**

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **12** de **10** del año **2026**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Tabla de Contenido

Contenido

Actividad Fase I – Investigación 8
Enunciado: 01. Arquitectura monolítica 8
Solución 8
Definición: 8
Problema que busca resolver: 8
Componentes principales: 8
Comunicación: 8
Ventajas: 8
Desventajas: 9
Ejemplo: 9
Proyecto apropiado: 9
Cuando no la recomendaría: 9
Fuente técnica: 9
Enunciado: 02. Arquitectura en capas (N-tier) 9
Solución 9
Definición: 9
Problema que busca resolver: 10
Componentes principales: 10
Comunicación: 10
Ventajas: 10
Desventajas: 10
Ejemplo: 10
Proyecto apropiado: 10
Cuando no la recomendaría: 11
Fuente técnica: 11
Enunciado: 03. Arquitectura cliente-servidor 11
Solución 11
Definición: 11

Problema que busca resolver: 11
Componentes principales: 11
Comunicación: 12
Ventajas: 12
Desventajas: 12
Ejemplo: 12
Proyecto apropiado: 12
Cuando no la recomendaría: 12
Fuente técnica: 12
Enunciado: 04. Arquitectura de microservicios 12
Solución 13
Definición: 13
Problema que busca resolver: 13
Componentes principales: 13
Comunicación: 13
Ventajas: 13
Desventajas: 13
Ejemplo: 14
Proyecto apropiado: 14
Cuando no la recomendaría: 14
Fuente técnica: 14
Enunciado: 05. Arquitectura orientada a servicios (SOA) 14
Solución 14
Definición: 14
Problema que busca resolver: 14
Componentes principales: 14
Comunicación: 15
Ventajas: 15
Desventajas: 15
Ejemplo: 15
Proyecto apropiado: 15
Cuando no la recomendaría: 15

Fuente técnica: 15
Enunciado: 06. Arquitectura dirigida por eventos (Event-Driven) 16
Solución 16
Definición: 16
Problema que busca resolver: 16
Componentes principales: 16
Comunicación: 16
Ventajas: 17
Desventajas: 17
Ejemplo: 17
Proyecto apropiado: 17
Cuando no la recomendaría: 17
Fuente técnica: 17
Enunciado: 07. Arquitectura serverless 17
Solución 18
Definición: 18
Problema que busca resolver: 18
Componentes principales: 18
Comunicación: 18
Ventajas: 18
Desventajas: 19
Ejemplo: 19
Proyecto apropiado: 19
Cuando no la recomendaría: 19
Fuente técnica: 19
Enunciado: 08. Arquitectura microkernel 19
Solución 19
Definición: 19
Problema que busca resolver: 20
Componentes principales: 20
Comunicación: 20
Ventajas: 20

Desventajas: 20
Ejemplo: 20
Proyecto apropiado: 20
Cuando no la recomendaría: 21
Fuente técnica: 21
Enunciado: 09. Arquitectura hexagonal 21
Solución 21
Definición: 21
Problema que busca resolver: 21
Componentes principales: 21
Comunicación: 21
Ventajas: 22
Desventajas: 22
Ejemplo: 22
Proyecto apropiado: 22
Cuando no la recomendaría: 22
Fuente técnica: 23
Enunciado: 10. Arquitectura peer-to-peer 23
Solución 23
Definición: 23
Problema que busca resolver: 23
Componentes principales: 23
Comunicación: 23
Ventajas: 24
Desventajas: 24
Ejemplo: 24
Proyecto apropiado: 24
Cuando no la recomendaría: 24
Fuente técnica: 24
Enunciado: 11. Arquitectura basada en componentes 24
Solución 25
Definición: 25

Problema que busca resolver: 25
Componentes principales: 25
Comunicación: 25
Ventajas: 25
Desventajas: 25
Ejemplo: 26
Proyecto apropiado: 26
Cuando no la recomendaría: 26
Fuente técnica: 26
Enunciado: 12. Arquitectura broker/pipeline 26
Solución 26
Definición: 26
Problema que busca resolver: 27
Componentes principales: 27
Comunicación: 27
Ventajas: 27
Desventajas: 28
Ejemplo: 28
Proyecto apropiado: 28
Cuando no la recomendaría: 28
Fuente técnica: 28
Actividad Fase ll – Análisis Comparativo 29
1. ¿Qué diferencia fundamental existe entre las cuatro? 29
2. ¿Cuáles permiten separar mejor las responsabilidades? Explica por qué 29
3. ¿Cuáles facilitan el despliegue independiente de partes del sistema? 30
4. ¿Qué arquitecturas requieren considerar especialmente la comunicación entre componentes? 30
5. ¿Cuál podría aumentar la complejidad de operación de un proyecto pequeño? 31
6. ¿Es posible combinar varios estilos de arquitectura en un mismo sistema? Justifique con un ejemplo 31
7. ¿Qué arquitectura elegiría para una aplicación con pocos usuarios y presupuesto limitado? ¿Cambiaria su elección si la aplicación tuviera millones de usuarios? Explique que otros factores evaluaría 32

Actividad Fase lll – Caso practico 33
A. Proponga una arquitectura inicial 33
Estilo o combinación de estilos seleccionados 33
Diagrama de los componentes principales 33
Responsabilidad de cada componente 33
Recorrido de los datos al registrar una cita 34
B. Analice dos alternativas 34
Alternativa 1: Arquitectura monolítica 34
Alternativa 2: Arquitectura de microservicios 35
C. Identifique riesgos 36
1. Fallo en el envío de notificaciones 36
2. Crecimiento de la cantidad de usuarios 36
3. Procesamiento pesado de reportes 36
D. Justifique su decisión 36
Actividad Fase lV – Evidencias del proceso 37
1. Registro de fuentes consultadas 37
2. Primer esquema o borrador del diagrama 37
3. Correcciones realizadas al borrador 37
4. Versión final del diagrama 38
5. Tabla de decisiones técnicas 38
6. Bitácora de dificultades 39
Dificultad 1: Elegir una arquitectura adecuada 39
Dificultad 2: Representar las notificaciones 39
Dificultad 3: Evitar una arquitectura demasiado compleja 39
7. Modificación entre el borrador y la versión final 39

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad Fase I – Investigación

Enunciado: 01. Arquitectura monolítica

Todo el sistema está construido como una sola aplicación

Solución

Definición:
La arquitectura monolítica consiste en construir una aplicación como una sola unidad. sus diferentes funcionalidades, como usuarios, productos, pagos o reportes, forman parte de una misma aplicación y normalmente se despliegan juntas

Problema que busca resolver:
Busca simplificar el desarrollo y despliegue de aplicaciones, especialmente cuando el proyecto no tiene un tamaño o complejidad muy grandes. al tener todo en una sola aplicación, el equipo no necesita administrar múltiples servicios independientes.

Por ejemplo, para una pequeña tienda virtual puede ser más sencillo tener usuarios, productos, carrito y ventas dentro de una misma aplicación

Componentes principales:

- Interfaz: permite al usuario interactuar con el sistema
- Lógica de negocio: contiene las reglas y procesos de la aplicación
- Acceso a datos: se encarga de consultar y modificar información
- Base de datos: almacena los datos del sistema

Aunque estos componentes pueden estar organizados en módulos, todos forman parte de la misma aplicación

Comunicación:
Los componentes normalmente se comunican mediante llamadas internas dentro de la aplicación. por ejemplo, cuando un usuario realiza una compra, la interfaz llama a la lógica de ventas y esta utiliza el acceso a datos para guardar la información

Ventajas:
Una ventaja es la simplicidad inicial**,** porque existe una sola aplicación que desarrollar y desplegar. esto resulta útil para equipos pequeños y proyectos con pocos módulos

Desventajas:
El principal problema aparece cuando la aplicación crece. una modificación en una parte puede obligar a reconstruir y desplegar toda la aplicación. Por ejemplo, si solamente se modifica el módulo de usuarios, pero la aplicación contiene también pagos, ventas y reportes, normalmente se termina desplegando nuevamente toda la aplicación

Ejemplo:
Un sistema web para administrar una pequeña tienda que contiene usuarios, productos, ventas e inventario dentro del mismo proyecto

Proyecto apropiado:
Sería apropiada para desarrollar un sistema de inventario para una pequeña empresa, porque el número de funcionalidades y usuarios puede ser relativamente reducido

Cuando no la recomendaría:
No la recomendaría para una plataforma muy grande con millones de usuarios y funcionalidades que necesiten escalar de manera independiente. en ese escenario, dividir el sistema podría facilitar el mantenimiento y escalamiento

Fuente técnica:
Microsoft azure architecture center

Dato consultado: Microsoft explica que los estilos de arquitectura tienen diferentes características y que los microservicios son adecuados cuando se necesita dividir funcionalmente un dominio complejo

Enunciado: 02. Arquitectura en capas (N-tier)

El sistema se divide en capas con responsabilidades diferentes

Solución

Definición:
La arquitectura en capas divide una aplicación en diferentes niveles, donde cada uno tiene una responsabilidad determinada.

Una organización común es:

Presentación => lógica de negocio => acceso a datos => base de datos

Microsoft describe n-tier como una arquitectura que divide una aplicación en capas lógicas y niveles físicos

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Problema que busca resolver:
Busca evitar que todo el código de una aplicación esté mezclado. al separar responsabilidades, resulta más sencillo saber dónde realizar cambios. Por ejemplo, si queremos cambiar la interfaz de un sistema, no deberíamos tener que modificar directamente la lógica que calcula las ventas

Componentes principales:

- Capa de presentación: Muestra información y recibe acciones del usuario.
- Capa de negocio: Contiene las reglas de la aplicación.
- Capa de acceso a datos: Realiza consultas y operaciones sobre los datos.
- Capa de datos: Almacena la información

Comunicación:
Normalmente una capa superior utiliza los servicios de una capa inferior.

Por ejemplo:

Usuario → presentación → negocio → datos → base de datos

Microsoft señala que una capa superior puede utilizar servicios de una capa inferior, pero una capa inferior no debería depender de una superior

Ventajas:
Una ventaja importante es la separación de responsabilidades. cada capa tiene una función definida, por lo que el código puede organizarse mejor. También facilita el mantenimiento. por ejemplo, podemos cambiar la forma de mostrar los datos sin modificar necesariamente las consultas a la base de datos

Desventajas:
Puede generar demasiadas capas y aumentar la complejidad. una operación sencilla puede tener que pasar por varias capas antes de llegar a la base de datos. Microsoft también señala que los sistemas grandes pueden hacer más difícil las pruebas y la supervisión debido a que las solicitudes pasan por varios niveles

Ejemplo:
Muchas aplicaciones empresariales tradicionales utilizan este tipo de organización.

Proyecto apropiado:
Un sistema académico del Sena, donde podríamos tener:

- presentación para los aprendices;
- lógica para las matrículas y cursos;
- acceso a datos;
- base de datos con aprendices, fichas y cursos.

Cuando no la recomendaría:
No sería mi primera opción para una aplicación que necesite modificar y desplegar funcionalidades de forma extremadamente independiente y frecuente, porque la estructura por capas puede generar dependencias entre diferentes partes

Fuente técnica:
Microsoft azure architecture center.

Dato consultado: n-tier divide la aplicación en capas lógicas y niveles físicos, y cada capa tiene una responsabilidad específica

Enunciado: 03. Arquitectura cliente-servidor

Un cliente solicita servicios y un servidor los proporciona

Solución

Definición:
Es una arquitectura donde existen dos partes principales: un cliente que solicita información o servicios y un servidor que recibe esas solicitudes y responde. Un ejemplo sencillo es un navegador web solicitando una página a un servidor

Problema que busca resolver:
Busca organizar la comunicación entre los usuarios y los recursos del sistema. En lugar de que cada computador tenga todos los datos y procesos, el servidor puede centralizar información y servicios

Componentes principales:

- Cliente: Realiza las solicitudes
- Servidor: Procesa las solicitudes
- Base de datos: Puede almacenar la información utilizada por el servidor
- Red: Permite la comunicación

Comunicación:
El cliente envía una solicitud al servidor y este devuelve una respuesta.

Por ejemplo:

navegador => solicitud http => servidor => respuesta http => navegador

Ventajas:
Una ventaja es que la información y lógica importante pueden centralizarse en el servidor. esto facilita administrar los datos desde un punto central. También permite que diferentes tipos de clientes utilicen el mismo servidor

Desventajas:
Si el servidor presenta una falla, los clientes pueden perder el acceso al servicio. Además, demasiadas solicitudes pueden sobrecargar el servidor si no está correctamente dimensionado

Ejemplo:
Una aplicación web donde los usuarios utilizan un navegador para conectarse a un servidor que procesa las solicitudes y consulta una base de datos

Proyecto apropiado:
Un sistema de gestión de aprendices, donde los computadores de los aprendices actúan como clientes y un servidor central administra los datos

Cuando no la recomendaría:
No sería la mejor opción para un sistema donde los dispositivos necesiten comunicarse directamente entre ellos sin depender constantemente de un servidor central

Fuente técnica:
Microsoft azure architecture center.

Dato consultado: Microsoft utiliza el concepto de cliente y diferentes componentes de procesamiento en sus estilos arquitectónicos, mostrando cómo las solicitudes pueden pasar desde el cliente hacia los niveles de la aplicación

Enunciado: 04. Arquitectura de microservicios

Una aplicación grande se divide en pequeños servicios independientes

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Solución

Definición:
Los microservicios dividen una aplicación grande en servicios pequeños e independientes, donde cada servicio se encarga de una función concreta.

Por ejemplo, una tienda virtual podría tener:

usuarios | productos | pagos | pedidos | notificaciones

Cada uno podría funcionar como un servicio independiente.

Microsoft describe los microservicios como servicios descompuestos funcionalmente que se comunican mediante apis y son adecuados para dominios complejos con actualizaciones frecuentes

Problema que busca resolver:
Busca evitar que una aplicación grande dependa de una única unidad de software. Sí tenemos un servicio de pagos que necesita muchos recursos, podemos escalarlo sin tener que escalar necesariamente el servicio de usuarios

Componentes principales:

- Microservicios: Cada uno realiza una función
- Apis: Permiten que los servicios se comuniquen
- Base de datos: Puede existir una o varias según el diseño
- Api gateway: Puede servir como punto de entrada para los clientes

Comunicación:
Los servicios pueden comunicarse mediante apis, normalmente utilizando protocolos como http/rest, aunque también pueden utilizar mensajería y eventos

Ventajas:
Permite escalar servicios individualmente. si el servicio de pagos recibe mucha carga, puede aumentarse su capacidad sin aumentar necesariamente la de los demás. También permite que diferentes equipos trabajen en diferentes servicios

Desventajas:
Aumenta la complejidad. en lugar de administrar una aplicación, ahora hay múltiples servicios, redes, despliegues, logs y posibles fallos de comunicación

Ejemplo:
Plataformas grandes de comercio electrónico, streaming y servicios digitales pueden utilizar arquitecturas basadas en microservicios

Proyecto apropiado:
Una plataforma de comercio electrónico grande, con servicios independientes para usuarios, productos, pagos, pedidos y envíos

Cuando no la recomendaría:
No la recomendaría para una aplicación pequeña creada por uno o dos desarrolladores. dividir un proyecto pequeño en muchos servicios podría agregar complejidad innecesaria

Fuente técnica:
Microsoft azure architecture center.

Dato consultado: microsoft indica que los microservicios dividen funcionalmente los servicios y permiten que estos se comuniquen mediante apis, siendo apropiados para dominios complejos y con actualizaciones frecuentes

Enunciado: 05. Arquitectura orientada a servicios (SOA)

Diferentes servicios reutilizables se comunican para formar aplicaciones

Solución

Definición:
Soa organiza el software alrededor de servicios reutilizables que proporcionan determinadas funciones empresariales. Por ejemplo, una empresa podría tener un servicio de pagos que puede ser utilizado por diferentes aplicaciones. Ibm explica que soa permite que los componentes sean reutilizables e interoperables mediante interfaces de servicio

Problema que busca resolver:
Busca evitar que diferentes aplicaciones tengan que programar nuevamente las mismas funciones. Por ejemplo, si una empresa ya tiene un servicio para consultar clientes, una nueva aplicación puede utilizar ese servicio en lugar de crear nuevamente todo el proceso

Componentes principales:
- Proveedor de servicio: Ofrece una funcionalidad

- Consumidor: Utiliza el servicio
- Interfaz/contrato: Define cómo utilizar el servicio
- Bus de servicios — esb**:** pPuede encargarse de integrar, enrutar y transformar comunicaciones

Comunicación:
Los servicios pueden comunicarse mediante protocolos como soap/http o rest/http. ibm señala ambos como mecanismos utilizados para exponer servicios

Ventajas:
La reutilización es una de sus principales ventajas. una misma función empresarial puede ser utilizada por diferentes aplicaciones

Desventajas:
Puede ser costosa y compleja de implementar y mantener. ibm señala que las implementaciones soa pueden presentar mayor complejidad y costos de implementación y mantenimiento

Ejemplo:
Una empresa bancaria podría tener servicios independientes para consultar clientes, procesar pagos y consultar créditos, que luego son utilizados por diferentes aplicaciones

Proyecto apropiado:
Un sistema empresarial grande donde diferentes aplicaciones necesitan utilizar las mismas funcionalidades

Cuando no la recomendaría:
No la recomendaría para un proyecto pequeño donde solo existe una aplicación y pocas funcionalidades reutilizables, porque implementar toda una estructura soa podría ser más complejo que el problema que intenta resolver

Fuente técnica:
IBM — what is service-oriented architecture

Dato consultado: los servicios soa utilizan interfaces comunes y contratos que permiten reutilizar funciones sin conocer cómo están implementadas internamente

Enunciado: 06. Arquitectura dirigida por eventos (Event-Driven)

Los componentes reaccionan cuando ocurre un evento

Solución

Definición:
Es una arquitectura donde los componentes reaccionan cuando ocurre un evento.

Un evento puede ser:

- Un usuario realizando una compra
- Un archivo cargándose
- Un pago completándose
- Un nuevo usuario registrándose

Amazon Web Services define tres componentes principales: productores, routers y consumidores de eventos

Problema que busca resolver:
Busca reducir la dependencia directa entre componentes. Por ejemplo, el servicio de pedidos puede informar que ocurrió una compra sin tener que conocer directamente qué servicios necesitan recibir esa información

Componentes principales:

- Productor: Genera el evento
- Router/bus: Recibe y dirige el evento
- Consumidor: Recibe y procesa el evento

Comunicación:
La comunicación se realiza mediante eventos y mensajes.

Ejemplo:

Compra realizada => evento "pedido_creado" => bus => inventario + notificaciones + facturación

Aws explica que el productor publica el evento y el router lo filtra y lo envía a los consumidores correspondientes

Ventajas:
Los componentes quedan desacoplados. el productor no necesita saber exactamente quién consumirá el evento. También puede ser muy útil cuando existen picos de actividad, porque los mensajes pueden procesarse de manera independiente

Desventajas:
Puede ser más difícil de depurar porque una acción puede desencadenar muchos eventos y procesos diferentes. Además, al utilizar comunicación asíncrona, obtener inmediatamente una respuesta de todos los procesos puede ser más complicado. aws señala precisamente que devolver resultados es más complejo en aplicaciones event-driven asíncronas

Ejemplo:
Un sistema de comercio electrónico puede generar un evento cuando se realiza un pedido. ese evento puede activar inventario, facturación y notificaciones

Proyecto apropiado:
Un sistema de notificaciones para una plataforma educativa, donde eventos como "aprendiz completó una lección" puedan activar diferentes acciones

Cuando no la recomendaría:
No la recomendaría para operaciones donde sea indispensable obtener una respuesta inmediata y determinista entre todos los componentes. Aws advierte que la comunicación basada en red introduce latencia variable y puede no ser adecuada para sistemas que necesitan latencias extremadamente bajas

Fuente técnica:
Aws — event-driven architecture.

Dato consultado: una arquitectura dirigida por eventos utiliza productores, routers y consumidores, manteniendo desacoplados los servicios

Enunciado: 07. Arquitectura serverless

El desarrollador ejecuta funciones sin administrar directamente los servidores

Solución

Definición:
La arquitectura serverless permite ejecutar código y utilizar servicios de infraestructura sin tener que administrar directamente los servidores.

Un ejemplo conocido es aws lambda, donde se ejecutan funciones cuando ocurre una solicitud o un evento

Problema que busca resolver:
Busca reducir la cantidad de infraestructura que los desarrolladores deben administrar.

En lugar de configurar y mantener servidores continuamente, el equipo puede concentrarse principalmente en desarrollar el código y utilizar los servicios necesarios.

Componentes principales:

- Funciones: Ejecutan pequeñas unidades de código
- Api gateway: Puede recibir solicitudes de los usuarios y enviarlas a las funciones
- Servicios de eventos: Pueden activar funciones cuando ocurre una determinada acción
- Base de datos o almacenamiento: Permite guardar la información utilizada por la aplicación

Comunicación:
Una solicitud o evento puede activar una función.

Por ejemplo:

Usuario => API Gateway => Lambda => Base de datos => Respuesta

De esta manera, el usuario realiza una solicitud, esta llega al servicio correspondiente y la función procesa la información necesaria

Ventajas:
Una ventaja importante es que el desarrollador no tiene que encargarse directamente de administrar los servidores. También puede facilitar el escalamiento, porque los recursos pueden aumentar o disminuir según la cantidad de solicitudes que reciba la aplicación.

Por ejemplo, una función que normalmente recibe pocas solicitudes puede utilizar más recursos cuando existe un aumento repentino de usuarios

Desventajas:
Una desventaja es que el sistema puede depender bastante del proveedor de servicios en la nube utilizado. También pueden existir restricciones relacionadas con el tiempo de ejecución, los recursos disponibles o la forma en que funcionan las funciones.

Ejemplo:
AWS Lambda es un ejemplo real de tecnología que permite ejecutar código utilizando un modelo serverless

Proyecto apropiado:
Sería apropiada para un sistema de generación automática de certificados. Por ejemplo, una función podría ejecutarse cuando un aprendiz termine un curso y generar automáticamente su certificado

Cuando no la recomendaría:
No la recomendaría para una aplicación que necesite mantener un servidor funcionando continuamente y que requiera un control completo sobre el sistema operativo o la infraestructura. En ese caso, podría ser más conveniente utilizar servidores tradicionales o máquinas virtuales

Fuente técnica:
AWS — Serverless Computing.

Dato obtenido: AWS explica que en la computación serverless el proveedor administra la infraestructura y asigna recursos cuando las funciones necesitan ejecutarse.

Enunciado: 08. Arquitectura microkernel

Existe un núcleo pequeño y se agregan funcionalidades mediante módulos

Solución

Definición:
La arquitectura microkernel utiliza un núcleo pequeño que contiene las funciones fundamentales del sistema y permite agregar otras funcionalidades mediante módulos o complementos. Una forma sencilla de entenderla es:

Núcleo principal + Plugins

Por ejemplo, un programa puede tener unas funciones básicas y permitir que el usuario instale plugins para agregar nuevas características

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Problema que busca resolver:
Busca facilitar la creación de sistemas que puedan ampliarse sin tener que modificar constantemente el núcleo principal. Esto permite agregar nuevas funciones mediante módulos independientes.

Componentes principales:

- Microkernel: Contiene las funciones esenciales del sistema
- Plugins o módulos: Agregan nuevas funcionalidades
- Interfaces: Permiten que los plugins se comuniquen correctamente con el núcleo

Comunicación:
Los plugins se comunican con el núcleo mediante interfaces previamente definidas. Por ejemplo, un programa puede establecer una interfaz para plugins de idiomas y posteriormente agregar un plugin para inglés, otro para español y otro para francés

Ventajas:
Una ventaja importante es la extensibilidad, porque se pueden agregar nuevas funcionalidades sin modificar completamente el núcleo del sistema. También permite que diferentes usuarios tengan diferentes conjuntos de funcionalidades dependiendo de los plugins que instalen

Desventajas:
Una desventaja es que diseñar correctamente las interfaces entre el núcleo y los plugins puede ser complicado. Además, un plugin mal desarrollado puede generar problemas de compatibilidad o afectar el funcionamiento del sistema

Ejemplo:
Eclipse es un ejemplo conocido de una plataforma extensible que utiliza plugins para agregar diferentes funcionalidades

Proyecto apropiado:
Sería apropiada para desarrollar un editor de código educativo. El programa podría tener funciones básicas para escribir código y permitir instalar plugins para diferentes lenguajes como JavaScript, Java o Python

Cuando no la recomendaría:
No la recomendaría para una aplicación muy pequeña que solamente necesita unas pocas funcionalidades fijas.

En ese caso, crear todo un sistema de plugins aumentaría la complejidad sin aportar un beneficio importante

Fuente técnica:
Eclipse Documentation.

Dato obtenido: Eclipse utiliza una arquitectura extensible mediante plugins, permitiendo agregar diferentes funcionalidades a la plataforma

Enunciado: 09. Arquitectura hexagonal

Separa la lógica del negocio de elementos externos como bases de datos o interfaces

Solución

Definición:
La arquitectura hexagonal busca separar la lógica principal del negocio de los elementos externos, como bases de datos, interfaces de usuario, APIs o servicios externos. También se conoce como Ports and Adapters, o puertos y adaptadores.

Una forma sencilla de representarla es:

Lógica de negocio → Puertos → Adaptadores → Elementos externos

Problema que busca resolver:
Busca evitar que la lógica principal del sistema dependa directamente de una tecnología específica. Por ejemplo, si una aplicación utiliza MySQL, la lógica de negocio no debería depender directamente de MySQL, porque posteriormente podríamos querer cambiar a otra base de datos.

Componentes principales:

- Núcleo de negocio: Contiene las reglas principales del sistema
- Puertos: Definen cómo el núcleo se comunica con elementos externos
- Adaptadores: Conectan los puertos con tecnologías concretas

Comunicación:
El núcleo utiliza los puertos para comunicarse con el exterior y los adaptadores implementan esas conexiones. Por ejemplo:

Lógica de negocio => Puerto => Adaptador MySQL => Base de datos

Si posteriormente se cambia MySQL por otra tecnología, se puede crear otro adaptador sin modificar necesariamente toda la lógica del negocio

Ventajas:
Una ventaja es que facilita las pruebas del sistema, porque los elementos externos pueden reemplazarse por componentes simulados. También facilita cambiar tecnologías externas.

Por ejemplo, si una aplicación cambia de MySQL a PostgreSQL, se puede modificar o reemplazar el adaptador correspondiente sin cambiar las reglas principales del negocio

Desventajas:
Requiere más organización y código inicial que una aplicación sencilla. Para un proyecto pequeño, crear diferentes puertos y adaptadores puede resultar innecesario y aumentar la complejidad.

Ejemplo:
Un sistema empresarial que utiliza diferentes proveedores de pago puede utilizar adaptadores para conectarse con cada proveedor sin modificar la lógica principal de los pagos

Proyecto apropiado:
Sería apropiada para un sistema de ventas que pueda utilizar diferentes bases de datos o proveedores de pago. Por ejemplo, el sistema podría utilizar inicialmente MySQL y posteriormente cambiar a PostgreSQL sin modificar directamente las reglas principales del negocio

Cuando no la recomendaría:
No la recomendaría para una página web académica muy sencilla que solamente tenga unas pocas funciones.

En ese caso, implementar toda la estructura de puertos y adaptadores podría generar más código y complejidad de la necesaria

Fuente técnica:
Alistair Cockburn - Hexagonal Architecture.

Dato obtenido: El patrón propone separar la aplicación de las tecnologías externas mediante puertos y adaptadores

Enunciado: 10. Arquitectura peer-to-peer

Los equipos se comunican directamente entre ellos sin depender necesariamente de un servidor central

Solución

Definición:
La arquitectura peer-to-peer permite que los dispositivos conectados a una red puedan comunicarse directamente entre ellos. Los dispositivos pueden actuar tanto como clientes como servidores. Esto significa que no necesariamente existe un único servidor central que controle todas las comunicaciones.

Problema que busca resolver:
Busca permitir que los dispositivos compartan recursos directamente entre ellos. Por ejemplo, un computador puede enviar un archivo directamente a otro computador y también recibir archivos de otros participantes.

Componentes principales:

- Peers: Son los dispositivos que participan en la red
- Red: Permite conectar los diferentes dispositivos
- Protocolos: Establecen las reglas para que los dispositivos puedan encontrarse y comunicarse

Comunicación:
Los dispositivos pueden comunicarse directamente entre ellos.

Por ejemplo:

Computador A => Computador B

Computador B => Computador C

De esta forma, los dispositivos pueden compartir información sin depender necesariamente de un servidor central

Ventajas:
Una ventaja es que la carga puede distribuirse entre diferentes dispositivos. También puede reducir la dependencia de un servidor central, porque los participantes pueden compartir recursos directamente

Desventajas:
Una desventaja es que administrar la seguridad puede ser más complicado. Además, un dispositivo puede desconectarse en cualquier momento, por lo que el sistema debe estar preparado para funcionar cuando algunos participantes no estén disponibles

Ejemplo:
BitTorrent es un ejemplo conocido de tecnología que utiliza comunicación peer-to-peer para compartir archivos. Los participantes pueden intercambiar diferentes partes de un archivo entre ellos

Proyecto apropiado:
Sería apropiada para una aplicación de intercambio de archivos entre computadores. Por ejemplo, varios estudiantes podrían compartir archivos directamente entre sus computadores dentro de una red

Cuando no la recomendaría:
No la recomendaría para un sistema bancario donde se necesite un control centralizado de las transacciones. En un sistema bancario sería importante mantener un control central sobre los datos, permisos y operaciones financieras.

Fuente técnica:
BitTorrent Protocol Documentation.

Dato obtenido: BitTorrent utiliza un modelo en el que los participantes pueden intercambiar partes de archivos entre sí mediante una red peer-to-peer

Enunciado: 11. Arquitectura basada en componentes

El sistema se construye utilizando componentes reutilizables

Solución

Definición:
La arquitectura basada en componentes construye una aplicación utilizando componentes independientes y reutilizables. Cada componente tiene una función determinada y puede utilizarse en diferentes partes del sistema.

Esto se parece bastante a lo que ya has trabajado en React, donde puedes crear componentes como navbar, sidebar, tabla y tarjeta

Problema que busca resolver:
Busca evitar la repetición de código y facilitar la construcción de sistemas mediante piezas reutilizables. Por ejemplo, si necesitamos mostrar muchas tarjetas iguales en una aplicación, podemos crear un componente de tarjeta y reutilizarlo varias veces

Componentes principales:

- Componentes: Contienen funcionalidades específicas
- Interfaces: Indican cómo utilizar los componentes
- Aplicación o contenedor: Integra los diferentes componentes

Comunicación:
Los componentes se comunican mediante mecanismos definidos por la tecnología utilizada. Por ejemplo, en React un componente padre puede enviar información a un componente hijo mediante props. También pueden comunicarse mediante eventos o funciones

Ventajas:
Una ventaja importante es la reutilización. Por ejemplo, si tenemos un componente de tarjeta utilizado diez veces, podemos modificar el componente una sola vez y aplicar el cambio en todos los lugares donde se utiliza. También facilita organizar proyectos grandes porque cada componente puede tener una responsabilidad determinada

Desventajas:
Una desventaja aparece cuando los componentes tienen demasiadas dependencias entre ellos. Por ejemplo, modificar un componente podría provocar problemas en otros componentes que dependen de él

Ejemplo:
React utiliza un modelo basado en componentes para construir interfaces de usuario.

Los componentes pueden reutilizarse y combinarse para crear interfaces más grandes

Proyecto apropiado:
Sería apropiada para un dashboard administrativo desarrollado en React.

Por ejemplo, podríamos tener:

- sidebar
- navbar
- tarjetas
- tabla
- botones

Cada elemento sería un componente que puede reutilizarse

Cuando no la recomendaría:
No sería necesario utilizar una arquitectura de componentes demasiado compleja para una página extremadamente sencilla y estática. En ese caso, crear muchos componentes podría aumentar el código sin aportar una ventaja significativa.

Fuente técnica:
React Documentation.

Dato obtenido: React permite construir interfaces mediante componentes, que son piezas reutilizables de la interfaz de usuario.

Enunciado: 12. Arquitectura broker/pipeline

Broker coordina comunicación entre componentes; pipeline procesa información mediante etapas

Solución

Definición:
En una arquitectura broker, existe un intermediario que recibe mensajes o solicitudes y los dirige hacia los componentes correspondientes. En una arquitectura pipeline, la información pasa por una serie de etapas, donde cada etapa realiza una operación antes de enviar el resultado a la siguiente.

Un ejemplo sencillo de pipeline sería:

Entrada => Validar => Procesar => Transformar => Salida

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Problema que busca resolver:
Busca organizar la comunicación y el procesamiento entre diferentes partes del sistema. En el caso del broker, los componentes no necesitan comunicarse directamente entre todos ellos porque existe un intermediario que puede dirigir los mensajes. En el caso del pipeline, un proceso complejo puede dividirse en diferentes etapas más pequeñas.

Componentes principales:
Broker:

- Productores: Generan los mensajes
- Broker: Recibe y dirige los mensajes
- Consumidores: Reciben y procesan los mensajes

Pipeline:

- Entrada: Recibe la información
- Etapas o filtros: Procesan o transforman la información
- Salida: Entrega el resultado final

Comunicación:
En un broker:

Componente A => Broker => Componente B

El broker recibe el mensaje y determina hacia dónde debe enviarlo.

En un pipeline:

Datos => Etapa 1 => Etapa 2 => Etapa 3 => Resultado

Cada etapa recibe el resultado de la anterior y realiza una operación específica

Ventajas:
Una ventaja del broker es que facilita el enrutamiento de mensajes.

Por ejemplo, un sistema puede enviar una solicitud al broker y este puede determinar qué servicio debe procesarla. En un pipeline, una ventaja es que un proceso complejo puede dividirse en pasos pequeños. Por ejemplo, un sistema de procesamiento de documentos puede separar la validación, extracción de información y almacenamiento en diferentes etapas.

Desventajas:
Un broker puede convertirse en un punto importante de dependencia.

Si el broker falla o recibe demasiadas solicitudes, puede afectar la comunicación entre los diferentes componentes.

En un pipeline, una etapa lenta puede retrasar las etapas posteriores.

Por ejemplo, si la segunda etapa tarda demasiado en procesar los datos, la tercera etapa tendrá que esperar

Ejemplo:
Un sistema de mensajería puede utilizar un broker para recibir eventos y enviarlos a diferentes servicios.

Por ejemplo, Amazon EventBridge permite recibir eventos y dirigirlos hacia diferentes destinos mediante reglas

Proyecto apropiado:
Sería apropiada para un sistema de procesamiento de documentos.

Por ejemplo:

Recibir documento => Validar => Extraer información => Procesar => Almacenar

Cada etapa tendría una función específica y el sistema podría utilizar un intermediario para dirigir los mensajes entre los diferentes componentes

Cuando no la recomendaría:
No utilizaría un pipeline para un proyecto donde no exista un proceso que pueda dividirse naturalmente en diferentes etapas.

Por ejemplo, para una página web sencilla con información estática sería innecesario implementar un sistema de procesamiento mediante pipeline, porque aumentaría la complejidad sin aportar un beneficio real

Fuente técnica:
AWS — Amazon EventBridge

Dato obtenido: EventBridge permite recibir eventos de diferentes productores y dirigirlos hacia destinos utilizando reglas, lo que permite desacoplar los componentes que producen y consumen eventos

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad Fase ll – Análisis Comparativo

para este análisis se seleccionaron cuatro arquitecturas: arquitectura monolítica, arquitectura en capas, arquitectura de microservicios y arquitectura serverless. Estas cuatro permiten comparar diferentes formas de organizar, comunicar, desplegar y escalar un sistema

1. ¿Qué diferencia fundamental existe entre las cuatro?

La diferencia fundamental está en la forma en que cada arquitectura organiza y administra las diferentes partes de una aplicación.

La arquitectura monolítica construye todo el sistema como una sola aplicación. Las funcionalidades como usuarios, productos, ventas y reportes forman parte de la misma unidad y normalmente se despliegan juntas.

La arquitectura en capas divide la aplicación en diferentes niveles con responsabilidades específicas. Por ejemplo: presentación, lógica de negocio, acceso a datos y base de datos. Esto permite organizar mejor las funciones del sistema.

La arquitectura de microservicios divide una aplicación grande en servicios pequeños e independientes. Cada servicio puede encargarse de una función específica, como usuarios, productos, pagos o pedidos.

La arquitectura serverless permite ejecutar funciones sin que el desarrollador tenga que administrar directamente los servidores. Las funciones pueden ejecutarse cuando reciben una solicitud o cuando ocurre un evento.

Por lo tanto, la principal diferencia es que la arquitectura monolítica concentra el sistema en una sola unidad, la arquitectura en capas lo organiza por responsabilidades, los microservicios lo dividen en servicios independientes y serverless ejecuta funciones sin administrar directamente los servidores

2. ¿Cuáles permiten separar mejor las responsabilidades? Explica por qué

Las arquitecturas que permiten separar mejor las responsabilidades son principalmente la arquitectura en capas y la arquitectura de microservicios.

La arquitectura en capas separa las funciones de la aplicación según su responsabilidad. La capa de presentación muestra información y recibe acciones del usuario, la capa de negocio contiene las reglas de la aplicación y la capa de acceso a datos realiza las consultas y operaciones sobre la información.

Los microservicios también permiten una separación clara, pero de una manera más independiente. Cada microservicio se encarga de una función concreta. Por ejemplo, un servicio puede encargarse de los pagos mientras otro administra los usuarios. Esto permite que cada parte tenga una responsabilidad específica.

Por esta razón, considero que la arquitectura en capas permite separar responsabilidades dentro de una misma aplicación, mientras que los microservicios permiten separar funcionalidades en diferentes servicios independientes

3. ¿Cuáles facilitan el despliegue independiente de partes del sistema?

La arquitectura que facilita principalmente el despliegue independiente de partes del sistema es microservicios.

Esto se debe a que cada servicio funciona de manera independiente. Por ejemplo, si se modifica el servicio de pagos, se puede actualizar ese servicio sin tener que modificar necesariamente los servicios de usuarios o productos. Además, cada servicio puede escalarse de acuerdo con la cantidad de solicitudes que recibe.

La arquitectura serverless también facilita trabajar con partes independientes del sistema, porque las funciones pueden ejecutarse de manera individual cuando reciben solicitudes o eventos.

En cambio, la arquitectura monolítica no facilita tanto este tipo de despliegue, porque las diferentes funcionalidades forman parte de una misma aplicación. Si se modifica una parte, normalmente es necesario reconstruir y desplegar nuevamente la aplicación completa

4. ¿Qué arquitecturas requieren considerar especialmente la comunicación entre componentes?

Las arquitecturas que requieren considerar especialmente la comunicación entre componentes son principalmente microservicios y serverless.

En microservicios, los diferentes servicios deben comunicarse mediante APIs y también pueden utilizar mensajería o eventos. Por esta razón, es importante considerar las conexiones entre servicios y los posibles fallos de comunicación.

En serverless, una solicitud o evento puede activar una función y esta puede comunicarse con otros servicios, como una base de datos. Por ejemplo: usuario => API Gateway => Lambda => base de datos => respuesta.

La arquitectura en capas también tiene comunicación entre sus partes, pero normalmente sigue una estructura más organizada, donde una capa superior utiliza los servicios de una capa inferior

5. ¿Cuál podría aumentar la complejidad de operación de un proyecto pequeño?

La arquitectura que podría aumentar más la complejidad de operación de un proyecto pequeño es microservicios.

Esto ocurre porque, en lugar de administrar una sola aplicación, se deben administrar varios servicios, redes, despliegues, registros y posibles problemas de comunicación.

Por ejemplo, para una aplicación pequeña desarrollada por uno o dos programadores, dividir el sistema en servicios independientes para usuarios, productos, pagos y notificaciones podría generar más trabajo de administración.

Por esta razón, para un proyecto pequeño sería más conveniente utilizar una arquitectura sencilla, como la arquitectura monolítica, especialmente si existen pocas funcionalidades y usuarios.

6. ¿Es posible combinar varios estilos de arquitectura en un mismo sistema? Justifique con un ejemplo

Sí, es posible combinar varios estilos de arquitectura en un mismo sistema, porque cada arquitectura puede utilizarse para resolver una necesidad diferente.

Por ejemplo, una plataforma educativa podría utilizar una arquitectura en capas para organizar la aplicación principal. Al mismo tiempo, podría utilizar microservicios para funcionalidades que necesiten trabajar de manera independiente, como las notificaciones.

También podría utilizar serverless para generar automáticamente un certificado cuando un aprendiz termine un curso. En este caso, un evento podría activar una función encargada de generar el certificado.

Por lo tanto, no es necesario utilizar una sola arquitectura para todo el sistema. Es posible combinar diferentes estilos dependiendo de las necesidades de cada parte, aunque se debe tener en cuenta que utilizar varios estilos también puede aumentar la complejidad del proyecto

7. ¿Qué arquitectura elegiría para una aplicación con pocos usuarios y presupuesto limitado? ¿Cambiaria su elección si la aplicación tuviera millones de usuarios? Explique que otros factores evaluaría

Para una aplicación con pocos usuarios y presupuesto limitado, elegiría una arquitectura monolítica, posiblemente organizada internamente mediante capas.

La arquitectura monolítica sería apropiada porque permite desarrollar y desplegar una sola aplicación. Esto reduce la necesidad de administrar múltiples servicios independientes y puede ser una buena opción para equipos pequeños y proyectos con pocas funcionalidades.

Si la aplicación tuviera millones de usuarios, cambiaría mi elección y consideraría principalmente una arquitectura de microservicios. Esta arquitectura permitiría separar las funcionalidades y escalar los servicios de manera independiente.

Por ejemplo, si el servicio de pagos recibe muchas más solicitudes que el servicio de usuarios, se podría aumentar la capacidad del servicio de pagos sin tener que aumentar necesariamente la capacidad de todos los demás servicios.

Sin embargo, no solamente tendría en cuenta la cantidad de usuarios. También evaluaría:

- presupuesto: cuánto dinero existe para infraestructura y mantenimiento
- cantidad de funcionalidades: un sistema pequeño no necesita necesariamente una arquitectura compleja
- equipo de desarrollo: es importante considerar la experiencia del equipo para administrar los servicios
- necesidad de escalamiento: algunas partes del sistema pueden necesitar más recursos que otras
- frecuencia de cambios: si diferentes funcionalidades se actualizan frecuentemente, puede ser útil separarlas
- complejidad del sistema: mientras más grande sea el sistema, mayor puede ser la necesidad de dividir sus responsabilidades
- mantenimiento: la arquitectura debe ser manejable para el equipo durante todo el desarrollo del proyecto

Mi comclusion, no existe una arquitectura que sea la mejor para todos los proyectos. La elección depende del tamaño del sistema, cantidad de usuarios, presupuesto, equipo disponible, necesidad de escalamiento, frecuencia de cambios y complejidad de las funcionalidades

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad Fase lll – Caso practico

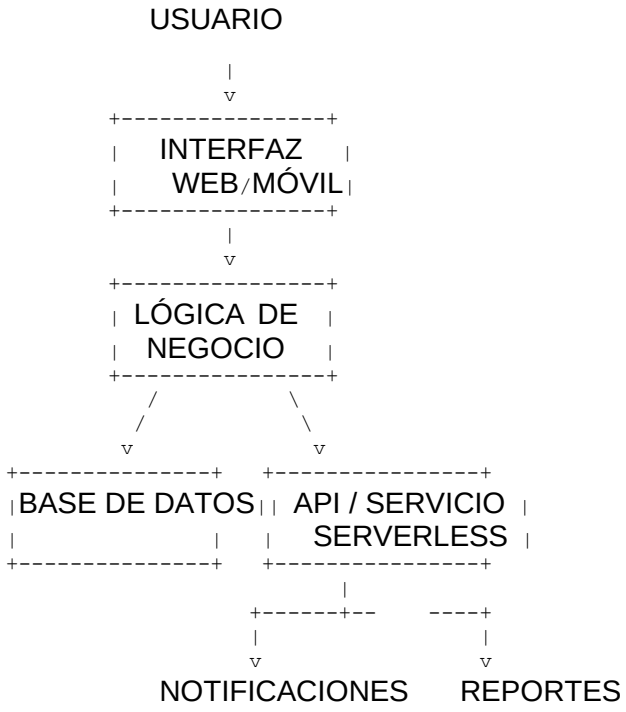
A. Proponga una arquitectura inicial

Estilo o combinación de estilos seleccionados
Para este sistema utilizaría una combinación de arquitectura en capas y arquitectura serverless.

La arquitectura en capas permitiría organizar la plataforma en diferentes responsabilidades, como presentación, lógica de negocio y acceso a datos. Esto facilitaría el mantenimiento y la organización del sistema.

La arquitectura serverless se utilizaría principalmente para las notificaciones automáticas y para algunas tareas que puedan ejecutarse de manera independiente, como el procesamiento de reportes. De esta manera, no sería necesario mantener servidores funcionando permanentemente para estas tareas.

Diagrama de los componentes principales



Responsabilidad de cada componente

- Interfaz: Permite que los usuarios registren citas, consulten información y reciban confirmaciones.
- Lógica de negocio: Se encarga de validar los datos de las citas, usuarios y demás reglas del sistema.

- Base de datos: Almacena usuarios, citas, notificaciones y la información necesaria para generar los reportes.
- Servicio serverless: Ejecuta funciones específicas cuando ocurre una acción o evento. Puede utilizarse para enviar notificaciones o procesar reportes.
- Notificaciones: Se encarga de enviar la confirmación de una cita y otras notificaciones automáticas.
- Reportes: Procesa la información almacenada para generar los reportes solicitados por la organización.

Recorrido de los datos al registrar una cita
El proceso podría funcionar de la siguiente manera:

1. El usuario: Ingresa a la plataforma y registra una nueva cita.
2. La interfaz: Envía los datos de la cita hacia la lógica de negocio.
3. La lógica de negocio: Valida que los datos sean correctos y verifica las reglas necesarias para registrar la cita.
4. La base de datos: Guarda la información de la nueva cita.
5. El evento: Después de registrar correctamente la cita, se puede generar un evento que active una función serverless.
6. La función serverless: Recibe el evento y ejecuta la tarea correspondiente para enviar la notificación.
7. La notificación: Se envía al usuario para confirmar que la cita fue registrada correctamente.

El recorrido sería:

Usuario => Interfaz => Lógica de negocio => Base de datos => Evento => Función serverless => Notificación => Usuario

De esta forma, el registro de la cita y el envío de la notificación pueden manejarse como procesos separados.

B. Analice dos alternativas

Alternativa 1: Arquitectura monolítica
Una alternativa sería utilizar una arquitectura monolítica, donde las funciones de usuarios, citas, notificaciones y reportes formen parte de una misma aplicación.

Ventajas:

- Sería sencilla de desarrollar al comienzo.

- Tendría una estructura fácil de administrar para un equipo pequeño.
- El despliegue sería más sencillo porque se manejaría una sola aplicación.
- Sería adecuada para la cantidad inicial de usuarios.

Dificultades:

- Si el sistema crece demasiado, toda la aplicación podría necesitar más recursos.
- Un cambio en una parte podría requerir desplegar nuevamente toda la aplicación.
- El módulo de reportes no tendría tanta independencia.
- Las notificaciones y otras funcionalidades estarían dentro de la misma aplicación.

Por estas razones, podría funcionar al principio, pero podría presentar dificultades cuando aumente el tamaño del sistema

Alternativa 2: Arquitectura de microservicios
Otra alternativa sería utilizar microservicios, separando el sistema en servicios independientes, por ejemplo: usuarios, citas, notificaciones y reportes.

Ventajas:

- Cada funcionalidad tendría una responsabilidad independiente.
- Los servicios podrían escalarse de manera individual.
- El módulo de reportes podría tener sus propios recursos.
- Las notificaciones podrían funcionar como un servicio independiente.
- Sería más apropiada si el sistema llegara a tener muchos usuarios.

Dificultades:

- Aumentaría la complejidad del proyecto.
- Los diferentes servicios tendrían que comunicarse mediante APIs, mensajes o eventos.
- Sería necesario administrar varios servicios y sus posibles fallos de comunicación.
- Para un sistema pequeño podría representar más trabajo del necesario.

Por esta razón, los microservicios serían una opción interesante si el sistema creciera considerablemente, pero podrían ser excesivos para la etapa inicial

C. Identifique riesgos

1. Fallo en el envío de notificaciones
Una notificación podría no enviarse correctamente después de registrar una cita.

Estrategia: Implementar mecanismos para detectar errores y permitir que la notificación vuelva a intentarse cuando sea necesario

2. Crecimiento de la cantidad de usuarios
La cantidad de usuarios podría aumentar y generar más solicitudes al sistema.

Estrategia: Utilizar servicios que puedan escalar de acuerdo con la demanda y separar las funciones que necesiten mayor capacidad

3. Procesamiento pesado de reportes
La generación de reportes podría consumir muchos recursos y afectar otras funciones de la plataforma.

Estrategia: Ejecutar el procesamiento de reportes mediante funciones o servicios independientes, evitando que este proceso afecte directamente el registro y consulta de citas

D. Justifique su decisión

Considero que la combinación de arquitectura en capas y serverless es adecuada para este sistema porque permite comenzar con una estructura organizada y sencilla, pero también proporciona posibilidades de crecimiento.

La arquitectura en capas permite separar las responsabilidades de la plataforma, facilitando el mantenimiento y la organización del código. Por otra parte, serverless permite ejecutar funciones específicas para tareas como las notificaciones automáticas y el procesamiento de reportes sin tener que administrar directamente los servidores.

Esta propuesta también permite que el sistema crezca progresivamente. No sería necesario comenzar directamente con una arquitectura de microservicios, ya que para pocos usuarios podría generar una complejidad innecesaria.

Sin embargo, si el número de usuarios aumentara considerablemente, si las diferentes funciones necesitaran escalar de manera independiente o si el sistema se volviera mucho más complejo, podría ser conveniente migrar algunas partes hacia una arquitectura de microservicios. En conclusión, la propuesta busca mantener una estructura sencilla al

inicio, pero dejando la posibilidad de separar y escalar las partes del sistema que lo necesiten a medida que la organización crezca

Actividad Fase IV – Evidencias del proceso

1. Registro de fuentes consultadas

Para realizar el análisis de las arquitecturas se consultaron las fuentes utilizadas en el material de estudio

Fuente	Tema consultado	Enlace	Fecha de consulta
Microsoft Azure Architecture Center	Arquitectura monolítica y arquitectura en capas	https://learn.microsoft.com/	12/10/2026
Microsoft Azure Architecture Center	Arquitectura de microservicios	https://learn.microsoft.com/	12/10/2026
AWS	Arquitectura serverless y funciones	https://aws.amazon.com/	12/10/2026
AWS	Arquitecturas basadas en eventos	https://aws.amazon.com/	12/10/2026

2. Primer esquema o borrador del diagrama

En el primer borrador se planteó una estructura sencilla en la que el usuario interactúa con la plataforma y esta se comunica directamente con la base de datos.

usuario => plataforma => base datos

Este primer esquema permitía representar la idea general del sistema, pero no mostraba claramente las diferentes responsabilidades ni el funcionamiento de las notificaciones y los reportes

3. Correcciones realizadas al borrador

Después de analizar las necesidades del sistema, se identificaron algunos elementos que debían agregarse.

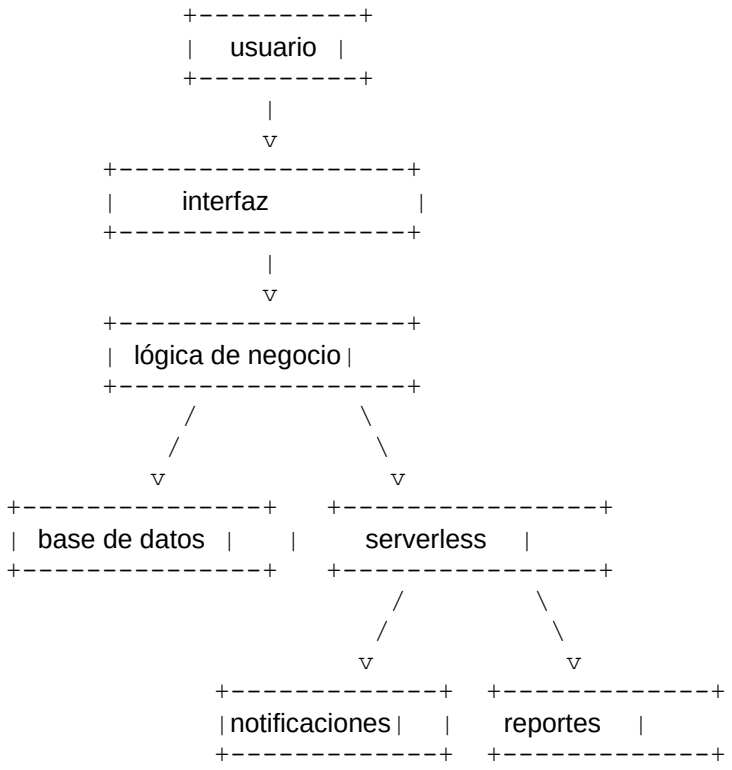
Primero: Se separó la plataforma en diferentes responsabilidades mediante una estructura en capas

Segundo: Se agregó un componente serverless para ejecutar tareas independientes

Tercero: Se separaron las notificaciones y los reportes como procesos específicos

Cuarto: Se agregó el recorrido de los datos desde el registro de una cita hasta el envío de la confirmación

4. Versión final del diagrama



La versión final representa mejor las responsabilidades de cada componente y muestra que las notificaciones y los reportes pueden ejecutarse de manera independiente mediante servicios serverless

5. Tabla de decisiones técnicas

Decisión tomada	Alternativa descartada	Justificación
Utilizar arquitectura en capas	Arquitectura monolítica sin separación interna	La arquitectura en capas permite organizar mejor las responsabilidades de la aplicación.
Utilizar serverless para notificaciones	Mantener las notificaciones dentro del proceso principal	Serverless permite ejecutar funciones específicas cuando ocurre una acción o evento.

Utilizar serverless para reportes	Procesar los reportes directamente dentro de la aplicación principal	Los reportes pueden requerir procesamiento independiente y podrían consumir recursos.
No utilizar microservicios inicialmente	Arquitectura de microservicios	El sistema comienza con pocos usuarios y utilizar microservicios desde el principio podría aumentar innecesariamente la complejidad.
Mantener una base de datos central	Separar inicialmente la información en varias bases de datos	Para la etapa inicial, una base de datos central facilita la administración del sistema.

6. Bitácora de dificultades

Dificultad 1: Elegir una arquitectura adecuada
Una de las dificultades fue determinar qué arquitectura sería conveniente para un sistema que comienza con pocos usuarios, pero que tiene posibilidades de crecimiento.

Solución: Se compararon diferentes arquitecturas y se decidió utilizar capas junto con serverless, buscando mantener una estructura sencilla y permitir que algunas funciones trabajen de manera independiente

Dificultad 2: Representar las notificaciones
Otra dificultad fue determinar cómo representar el proceso de envío automático de notificaciones.

Solución: Se decidió utilizar una función serverless que pueda activarse después de registrar correctamente una cita y encargarse del envío de la confirmación

Dificultad 3: Evitar una arquitectura demasiado compleja
También se consideró utilizar microservicios desde el comienzo, pero esto podía aumentar la complejidad para un sistema que inicialmente tendría pocos usuarios.

Solución: Se decidió comenzar con una arquitectura más sencilla y dejar abierta la posibilidad de utilizar microservicios en el futuro si el crecimiento del sistema lo requiere

7. Modificación entre el borrador y la versión final

La principal modificación entre el borrador y la versión final fue la separación de responsabilidades.

En el borrador, la plataforma aparecía como un único componente conectado directamente con la base de datos. En la versión final, se agregaron la interfaz, la lógica de negocio y el componente serverless.

Además: Las notificaciones y los reportes fueron separados como procesos independientes.

Esta modificación permite representar mejor la arquitectura propuesta y facilita comprender cómo se procesan los datos dentro del sistema